Exercise 1: Spring Data JPA

Business Scenario

Develop a Spring Boot application using Spring Data JPA and Hibernate to connect to a MySQL database. Create a Country entity, retrieve country details from the database, and display the records in the console.

Source Code

application.properties

```
# Spring Framework and application log
logging.level.org.springframework=info
logging.level.com.cognizant=debug
# Hibernate logs
logging.level.org.hibernate.SQL=trace
logging.level.org.hibernate.orm.jdbc.bind=trace
# Log pattern
logging.pattern.console=%d{dd-MM-yy} %d{HH:mm:ss.SSS} %-20.20thread %5p %-
25.25logger{25} %25M %4L %m%n
# Database configuration
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
spring.datasource.username=root
spring.datasource.password=********
# Hibernate configuration
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

Country.java

```
package com.cognizant.ormlearn.model;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
@Entity
@Table(name = "country")
public class Country {
   @Id
   @Column(name = "co_code")
```

```
    private String code;
    @Column(name = "co_name")
    private String name;
    public Country() {
    }
    public Country(String code, String name) {
        this.code = code;
        this.name = name;
    }
    public String getCode() {
        return code;
    }
    public void setCode(String code) {
        this.code = code;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}
```

CountryRepository.java

```
package com.cognizant.ormlearn.repository;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Country;
@Repository
public interface CountryRepository extends JpaRepository<Country, String> {
}
```

CountryService.java

```
package com.cognizant.ormlearn.service;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;
@Service
public class CountryService {
   @Autowired
   private CountryRepository countryRepository;
   public List<Country> getAllCountries() {
      return countryRepository.findAll();
   }
}
```

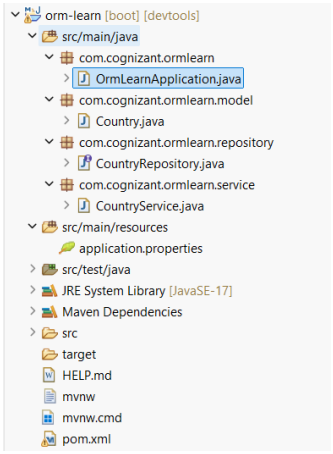
OrmLearnApplication.java

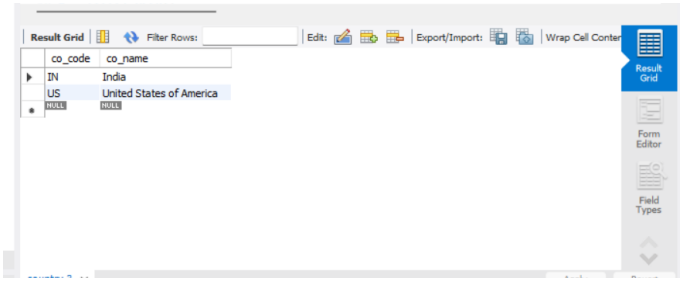
```
package com.cognizant.ormlearn;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import com.cognizant.ormlearn.service.CountryService;
@SpringBootApplication
public class OrmLearnApplication implements CommandLineRunner {
   @Autowired
   private CountryService countryService;
   public static void main(String[] args) {
      SpringApplication.run(OrmLearnApplication.class, args);
   }
   @Override
   public void run(String... args) throws Exception {
      System.out.println("Country List");
      countryService.getAllCountries().forEach(System.out::println);
   }
}
```

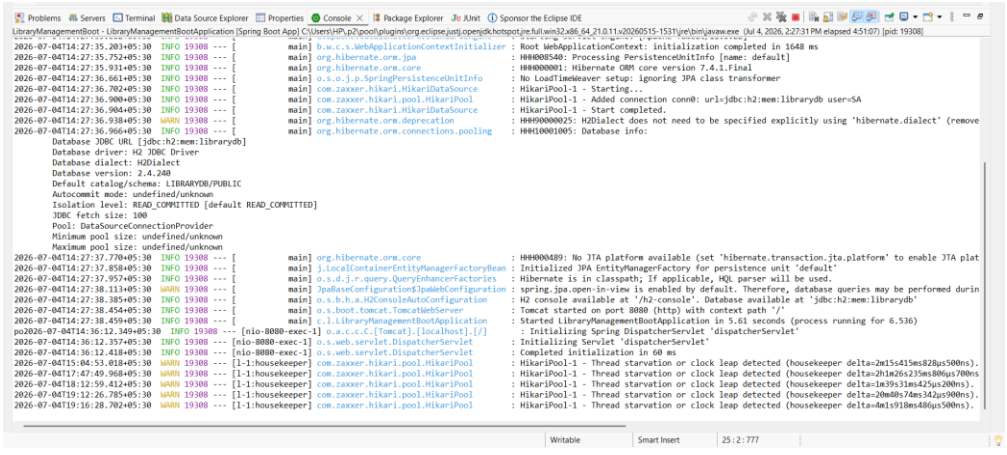
Database Script

```
CREATE DATABASE ormlearn;
USE ormlearn;
CREATE TABLE country (
   co_code VARCHAR(2) PRIMARY KEY,
   co_name VARCHAR(50)
);
INSERT INTO country VALUES ('IN', 'India');
INSERT INTO country VALUES ('US', 'United States of America');
SELECT * FROM country;
```

Output







Result

The Spring Boot application was successfully developed using Spring Data JPA and Hibernate. The application connected to the MySQL database, retrieved the country details from the country table, and displayed the records successfully in the console.

Exercise 2: Difference between JPA, Hibernate and Spring Data JPA

JPA (Java Persistence API) is a specification that provides a standard way to connect Java applications with relational databases. It defines the rules and annotations used for database operations, but it does not contain any implementation. Because of this, JPA cannot perform database operations by itself.

Hibernate is an ORM (Object Relational Mapping) framework that implements JPA. It converts Java objects into database records and database records into Java objects automatically. Hibernate also handles SQL query generation, transactions, and database communication, making database operations easier for developers.

Spring Data JPA is built on top of JPA and uses Hibernate internally. It reduces the amount of code that developers need to write. Instead of writing separate DAO classes and SQL queries for basic operations, we can simply create a repository interface by extending JpaRepository. It provides ready-made methods such as save(), findAll(), findById(), and deleteById(), making application development faster and simpler.

In simple words, **JPA defines the rules, Hibernate implements those rules, and Spring Data JPA makes using Hibernate much easier by reducing boilerplate code.** All three work together to simplify database development in Java applications.